

NAIRR Teaching and Curriculum Innovation

Cohort 3 · July 26 – August 1, 2026

Morehouse Supercomputing Facility • Building HBCU Courses with AI & HPC

[Full Program Overview](#)

Program at a Glance

Day	Date	Focus
Pre-Event	Sun, Jul 26	Account setup and software installs
Day 1	Mon, Jul 27	Tour of NAIRR; set up repos & environment; draft learning outcomes
Day 2	Tue, Jul 28	Curriculum map; syllabus draft; HPC with Vista and Tapis; module builder pt. 1; faculty panels
Day 3	Wed, Jul 29	Dataset discovery; quiz & assignment material; Jupyter on HPC; grading scale & rubrics
Day 4	Thu, Jul 30	Case study; exam scaffold; finish your module
Day 5	Fri, Jul 31	Publish course site; office hours setup; 90-day plan
Departure	Sat, Aug 1	Final checklist and travel home

Program & Team



KG

Dr. Kinnis Gosha

Program Lead; Executive
Director, Morehouse
CBPC



RJ

Dr. Renée Jordan

Instructor; Associate
Director



WN

Prof. Whitney Nelson

Instructor; Research
Scientist



AS

Dr. Ashley Scruse

Instructor; Deputy
Director,
Supercomputing Facility



CD

Cerrice Dawson

Program Coordinator

Who this is for

- Open to instructors across every discipline and skill level; no prior coding experience required
- If you code: your AI coding assistant speeds up writing and debugging the scripts your students run on HPC
- If you don't code: your AI coding assistant writes those scripts for you, in plain English

The Two-Repo Model, At a Glance

course-site (PUBLIC)

Everything a student will eventually see:
syllabus, schedule, modules, resources.

course-toolkit (PRIVATE)

Grading, exams, attendance, real prep notes.
Never gets published.

Both repos are cloned as sibling folders and opened together. Your AI coding assistant reads CLAUDE.md and skills/ in course-site for context, then writes into both repos as you work through each module this week.

Pre-Event: Before You Arrive

- ☐ Request your Morehouse Supercomputing Facility account (.edu email; approval can take a few business days); set up MFA
- ☐ Choose your setup: Option A — Git + VS Code/Antigravity + Claude Code, or Option B — GitHub Desktop + Claude Desktop (no IDE)
- ☐ Create a free GitHub account (or confirm your institution's GitHub Education org)
- ☐ Create a free Calendly account (optional, needed Day 3)
- ☐ Upload your current syllabus (or the model syllabus) plus your institution's policy checklist
- ☐ Check with your LMS admin whether Canva–Blackboard LTI is enabled, if you want it live this week

DAY 1

Set Up | GitHub Course Builder

Monday, July 27

Day 1: Today's Schedule

Time	Session
9:30–10:00	Arrival, coffee, informal networking
10:00–10:30	Welcome, daily agenda review & introductions
10:30–12:30	What is NAIRR? Tour of the ecosystem. Set up course-builder repos
12:30–1:30	Lunch
1:30–2:00	What goes into AI & a shared repo; what's safe, what's not
2:00–2:15	Scale-first design: creating assignments & courses for HPC; why HPC?
2:15–3:00	Creating and mapping your learning outcomes
7:00–8:00 PM	All White Party (evening)

What Is NAIRR?

- The National Artificial Intelligence Research Resource (NAIRR) Pilot connects researchers and educators to compute, cloud, and AI resources they wouldn't otherwise have access to
- This accelerator introduces you to NAIRR as one of the resources you'll use this week: five days building a real HPC- and AI-scaffolded course, using your AI coding assistant to do the technical heavy lifting
- NAIRR also offers ongoing access beyond this week; AI credits, compute time, and continued programs for qualifying researchers and educators — see nairrpilot.org
- Today's goal: understand the ecosystem you're now part of, then start building inside it

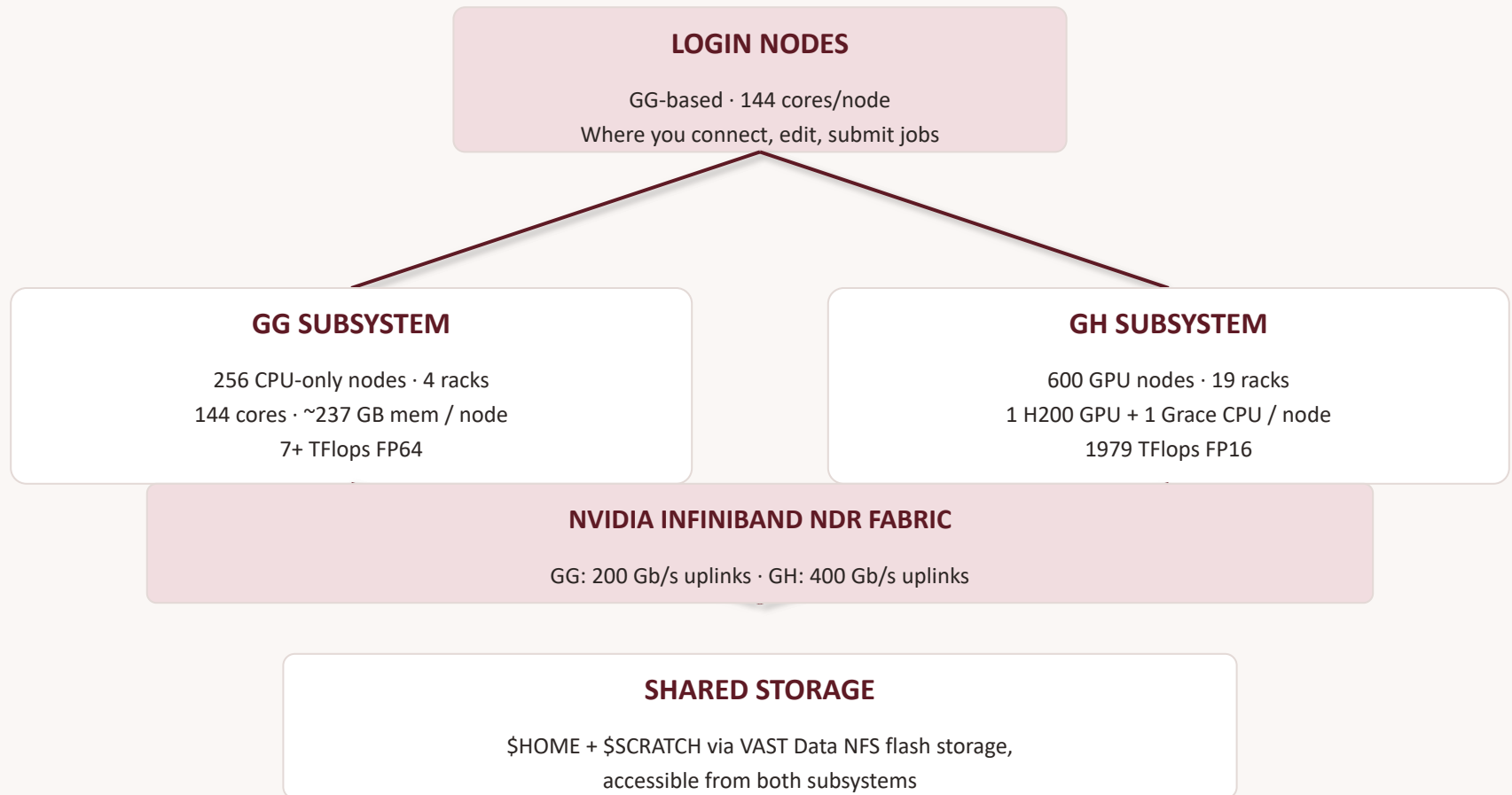
What Is Supercomputing? Why TACC / Vista Matters

- A supercomputer isn't one very fast computer — it's thousands of processors working together on the same problem at once, coordinated by scheduling software
- The Morehouse Supercomputing Facility's allocation runs on the Vista system at the Texas Advanced Computing Center (TACC), one of the most powerful academic research-computing centers in the world
- **Not all processors are the same: CPUs handle general-purpose, sequential work; GPUs repeat one simple calculation across thousands of parallel threads at once — exactly what training a neural network needs**
- Reach for CPU: a statistical regression on economic data, a rule-based search across case files, most spreadsheet-style or survey analysis
- Reach for GPU: training a sentiment classifier on student writing, an image classifier for forensic evidence, fine-tuning a language model
- Everything flagged "HPC-based" this week ultimately runs on one of Vista's two node types — more on that next

Vista System Topology

- Vista is organized as a two-level fat-tree network connecting two compute subsystems, plus shared storage
- **Login nodes — where you land, edit files, and submit jobs; GG-based, same 144-core architecture as GG compute nodes. You never run heavy computation here.**
- GG subsystem (Grace-Grace): 256 CPU-only nodes across 4 racks, 144 cores each, ~237 GB memory per node
- GH subsystem (Grace-Hopper): 600 nodes across 19 racks, each pairing one NVIDIA H200 GPU (96 GB) with one Grace CPU (72 cores)
- Both subsystems connect via NVIDIA InfiniBand NDR; GH nodes get double the network bandwidth of GG (400 vs. 200 Gb/s)

Vista Topology — At a Glance



Original schematic based on published specs; see docs.tacc.utexas.edu/hpc/vista/#system-topology for TACC's authoritative diagram.

Storing Data on Vista: Why It's Worth It

- Enterprise-grade, redundant storage — not a single laptop hard drive that can fail, get lost, or run out of space mid-semester
- Storage sits right next to the compute — no repeated uploading and downloading between where your data lives and where it actually gets processed
- Capacity far beyond a laptop: datasets too large to fit on personal hardware are routine here, not an exception
- Persistent across sessions — your data and environment are still there the next time you log in, no re-setup required
- Institutionally backed and secured — appropriate for the kind of research and student data this program takes seriously protecting
- Where exactly to put what — home vs. scratch vs. work — comes Day 2, once you're actually logging in and running jobs

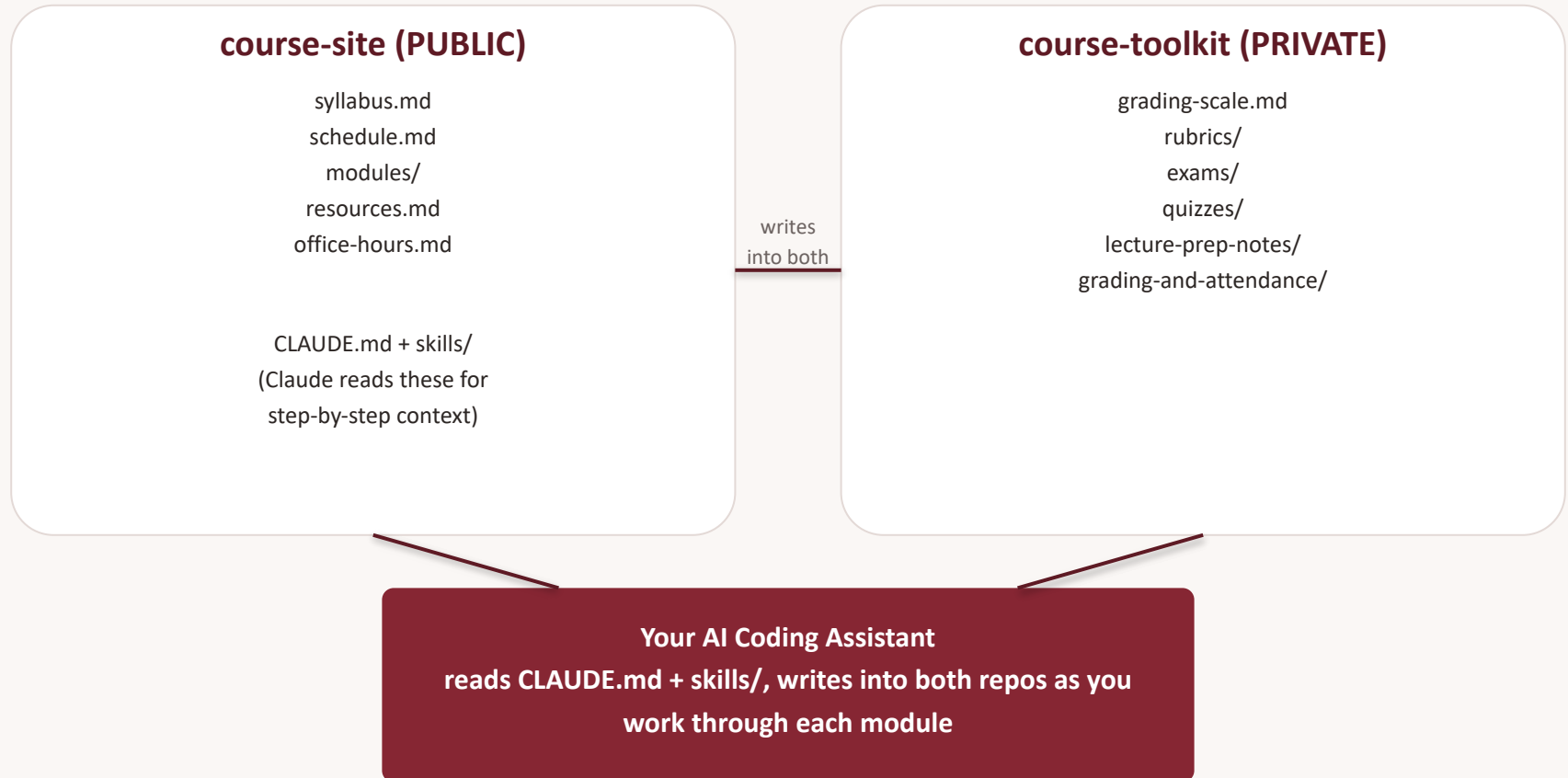
Good Conduct on Vista

- The core principle: what you do on the resource affects hundreds of other users sharing it
- Never run jobs or intensive computation on login nodes — use batch queues or an idev session
- Avoid I/O-heavy work in \$WORK; stage through \$SCRATCH instead, and use .tar archives instead of transferring many small files
- All AI-assisted workloads must run on compute nodes; you're responsible for everything an AI agent launches on your behalf, and all Service Units consumed charge to your allocation
- **Consequence: “Your account may be suspended if your jobs are impacting other users.”**

File Structure for AI: The Two-Repo Model

- Every course you build this week lives in two GitHub repos, opened together: course-site (public) and course-toolkit (private)
- This isn't just storage — it's the environment your AI coding assistant works inside all week
- course-site ships with a CLAUDE.md (root operating instructions) and a skills/ folder (step-by-step guides); your assistant reads both automatically for context before each module
- A well-documented repo — clear READMEs, decisions written down, explicit public/private boundaries — gives your AI coding assistant the same context a new human collaborator would need
- The more your repo documents itself, the less you have to re-explain in every session

The Two-Repo Model



- Create both repos from the template (“Use this template”); course-site → Public, course-toolkit → Private
- **Option A: Git + VS Code or Antigravity, then the Claude Code extension (or CLI fallback)**
- **Option B: GitHub Desktop for cloning/committing, Claude Desktop with the Filesystem connector enabled for reading/writing files — no IDE, no terminal**
- Clone both repos as sibling folders, opened together — every later module assumes this layout
- Both paths reach the same place by the end of the week; pick based on comfort with a code editor, not on coding ability itself

- Send one test prompt to confirm the connection: coders can ask it to explain a file in the repo; non-coders can ask for a one-line Slurm script with an explanation
- Checkpoint before lunch:
- ✓ course-site (public) and course-toolkit (private) created and cloned
- ✓ Both folders open together
- ✓ Claude connected — Code (Option A) or Desktop + Filesystem connector (Option B)
- ✓ Test prompt sent successfully

What Goes Into AI & a Shared Repo: What's Safe

- course-site is PUBLIC the moment GitHub Pages is turned on — treat everything you paste into it, or into a prompt that writes to it, as visible to the world
- Safe to share: syllabus language, schedule/topics, published assignment prompts, public datasets, your own original lecture material
- Course-toolkit stays private — grading keys, rubrics, real student data, and prep notes belong there, never in course-site
- When in doubt, draft in course-toolkit first and move only the public-safe version over

What Goes Into AI & a Shared Repo: What's Not Safe

- Never paste identifiable student data into any AI tool, public or private, without de-identifying it first (name, ID, email removed or replaced)
- Answer keys, rubric scoring detail, and grading weights never belong in course-site
- This framing carries through the whole week — it resurfaces directly in Day 3's AI-use rubric and Day 5's pre-publish leak check

Scale-First Design: Why HPC?

- Not every assignment needs a supercomputer — the goal today is learning to tell the difference, not defaulting to HPC because it's available
- HPC earns its place when an assignment needs scale (a dataset too big for a laptop), speed (a simulation that would take hours becomes minutes), or parallelism (the same computation repeated across many configurations at once)
- If a task runs fine locally, the HPC framing should move to a different part of the course, not be forced onto this one

The Framing Question

- Before building any HPC-based assignment this week, answer this in one sentence:
- **“What specifically requires HPC here — scale, speed, or parallelism?”**
- Can't answer it in one sentence? The assignment likely works fine on a laptop — and that's a useful finding, not a failure
- This question comes back explicitly in the Sample Assignments resource and in every Module Builder session this week

Creating and Mapping Your Learning Outcomes

Draft 3–5 learning outcomes for your course, one at a time — at least one must involve using HPC resources. For each, note the assessment type and whether it's HPC-based.



“Help me write learning outcomes for a [course topic] course. Students range from [prior experience]. I want at least one outcome that involves using HPC resources.”

Already have outcomes drafted? Paste them in and ask your AI coding assistant to map them against the HPC requirement instead of starting from scratch. Feeds: [course-toolkit/learning-outcomes-map.md](#).

DAY 2

HPC Acclimation + Curriculum Map

Tuesday, July 28

Day 2: Today's Schedule

Time	Session
10:00–10:15	Daily agenda review
10:15–11:00	Build your curriculum map & schedule
11:00–12:00	Faculty Fellows Panel I: Designing AI Learning Experiences Across Disciplines
12:00–12:30	Draft your syllabus
12:30–1:30	Lunch
1:30–2:00	HPC with Vista: architecture, filesystem, and SSH
2:00–2:30	Module 1: Getting acclimated via TAP
2:30–3:00	Module Builder, Part 1: lecture outline + slide deck
3:00–4:00	Faculty Fellows Panel II: Implementing, Evaluating, and Scaling AI Education
5:30 PM	Cielo Beach Club (evening)

Working from yesterday's learning outcomes, sequence your course week by week (or module by module), noting where the HPC-based assignment should land.



“Given these learning outcomes [paste table], suggest a logical order across N weeks, noting where the HPC-based assignment should land.”

Then publish the public-facing version — topic and due dates only, no grading detail — into course-site/schedule.md.

Faculty Panel I: Designing AI Learning Experiences Across Disciplines

- 11:00 AM – 12:00 PM
- Dr. Feseha Abebe-Akele — Assistant Professor of Genetics, Elizabeth City State University
- Dr. Edgar J. Lobaton — Director, Applied AI Initiative in Engineering & CS, NC State University
- Dr. Oyebade Kunle Oyerinde — Special Assistant to the Provost for AI, Data & Analytics, Clark Atlanta University
- Dr. Diane Uwacu — Assistant Professor, Computer Science, Mount Holyoke College

A 30-minute session that produces a draft skeleton, not a finished syllabus. Start from your existing syllabus or the model syllabus provided.



“Adapt this model syllabus for a [course topic] course with these learning outcomes: [paste from learning-outcomes-map.md].”

Pull in your outcomes summary and schedule link. Leave explicit placeholders for HPC access language and grading policy — both get filled in later this week.

HPC with Vista: Login vs. Compute Nodes, in Practice

- You'll land on a login node every time you connect — this is for editing files, managing jobs, and light tasks only
- Actual computation always happens on compute nodes, reached through a batch job (sbatch) or an interactive session (idev) — never by running heavy work directly on login
- This is a policy, not just a suggestion — running compute-intensive work on a login node affects everyone else connected to it at the same time
- Today's goal: get comfortable with that boundary before Wednesday's hands-on Jupyter and job-monitoring sessions

HPC with Vista: Filesystem Layout

- Three directories, three purposes — knowing which one to use prevents most first-week HPC mistakes
- **\$HOME** — small and persistent. Config files, small personal scripts. Not for job data.
- **\$SCRATCH** — large, fast, temporary. Stage your data here, run the job, move results out before the 10-day purge.
- **\$WORK** — shared across the whole system. Reserve for project-level storage, not active I/O — heavy use here slows down every other TACC user, not just you.
- Rule of thumb for this week's assignments: dataset lands in \$SCRATCH, job runs from \$SCRATCH, results get copied out before you're done

HPC with Vista: Connecting via SSH

- Most of this week's access happens through your browser and Tapis — covered in Module 00–01 and this afternoon's TAP session — so you won't need to hand-configure SSH just to get started
- If your program setup calls for a direct SSH client: Mac/Linux terminals have one built in; Windows users can use PuTTY or the Git Bash terminal that ships with Git for Windows
- Confirm your exact login endpoint with your program contact — this can vary by cohort and shouldn't be guessed at
- Full path from account request to first login: see “Accessing the Morehouse Supercomputing Facility via TAPIS”

- TAP (the TACC Analysis Portal) is the point-and-click way to get an interactive session on a compute node — no command line required
- Think of it as the browser equivalent of running `idev` from the terminal — same interactive session, nothing to type
- **Why TAP specifically, not raw SSH/command line:**
- Easier for non-coders: no Linux commands, no manual SSH session to configure; log in, pick Vista, and launch
- Easier for students: the same simplicity extends to your class, not just to you; a student who's never touched a terminal can still submit a real HPC job
- A clean UI: TAP gives you a web-based interface for launching and tracking interactive sessions, instead of a bare terminal prompt with no visual feedback
- This afternoon is hands-on: you'll walk through an actual example rather than reading about the concepts in the abstract

- 1. Log in at tap.tacc.utexas.edu with your TACC username, password, and MFA
- 2. Select Vista as the system, then choose New Session
- 3. Configure: application = idev, your allocation, the development queue, 1 node, a short time limit
- 4. Submit, wait for it to move from queued to running, then click Connect for a live terminal on the compute node
- Reference: Launching an idev Session on the TAP Analysis Portal

Pick one module from your curriculum map to build fully, end to end — the repeatable pattern you'll copy for every other module later.

“Draft a lecture outline for [module topic], covering [outcome]. Include a short intro, 2-3 core concepts, and a wrap-up tying to the HPC assignment.”

Split public vs. private: outline + slides link go in course-site/modules/module-XX.md; full prep notes stay in course-toolkit/lecture-prep-notes/.

Faculty Panel II: Implementing, Evaluating, and Scaling AI Education

- 3:00 – 4:00 PM
- Dr. Debanjali Banerjee — Assistant Professor of Computer Science, Earlham College
- Dr. Mia Champion — Assistant Teaching Professor, Computing & Software Systems, University of Washington Bothell
- Dr. Vanessa D'Amario — Assistant Professor, Nova Southeastern University
- Dr. Ananya Jana — Assistant Professor, Computer Science, Marshall University

DAY 3

AI for Calculations, Grading Scale + Module Builder Continues

Wednesday, July 29

Day 3: Today's Schedule

Time	Session
10:00–10:15	Daily agenda review
10:15–10:30	Dataset discovery for HPC; where to find data in your domain
10:30–11:30	Accessing Morehouse TAPIS, Using an AI coding assistant for API data extraction, moving data onto HPC
11:30–12:30	Module Builder, Part 2: quiz + assignment supplementary material
12:30–1:30	Lunch
1:30–2:30	HPC Acclimation Module 2: Jupyter via TAP (idev). Module 3: monitoring jobs & output
2:30–3:00	Build your grading scale & rubrics
Evening	Souvenir shopping

Dataset Discovery: Where to Look

- General repositories: data.gov, Kaggle Datasets, Google Dataset Search, Hugging Face Datasets
- Discipline-specific: genomics (NCBI/GEO), climate (NOAA), social science (ICPSR), astronomy (NASA/MAST), economics/finance (FRED, World Bank, SEC EDGAR), education (NCES, IPEDS)
- Ask your AI coding assistant: “What are the standard public dataset repositories for [my field]?”
- NAIRR-affiliated resources: ask your program contact about NAIRR's own resource index by discipline

Dataset Discovery: Scoping a Computational Plan

Once you have a candidate dataset, don't jump straight into scripting — describe the shape of the problem first and let your AI coding assistant help plan.

II

“I have a dataset of [size, format, contents]. My assignment goal is for students to [task]. What's a reasonable step-by-step computational workflow, and which steps actually need HPC vs. which could run on a laptop?”

Not every step needs a supercomputer — this identifies the one or two genuinely heavy steps so your HPC component is meaningful, not incidental.

Moving Data onto HPC: Transfer Options

- **scp** — small-to-medium files. Built into every Mac/Linux terminal and Git Bash on Windows.
- **rsync** — larger or repeated transfers. Only copies what's changed, resumes if interrupted.
- **Globus** — genuinely large datasets (10s of GB+). Web-based, automatic retry on drop.
- Common pitfalls: path typos silently create a new folder instead of erroring; check disk quota before a big transfer; confirm a transfer is complete before submitting a job that reads it

Moving Data onto HPC: Running the Job

- Once your data is on the Morehouse Supercomputing Facility, you submit jobs to process it — `idev` for interactive sessions, `sbatch`/Slurm for batch jobs
- If you don't code: describe to your AI coding assistant what you want the script to do with your uploaded dataset, and ask it to write the Slurm submission script, explained line by line
- This is the technical backbone for the assignment supplementary material you build next in Module Builder, Part 2

- Build the quiz with a full answer key — stays private in course-toolkit/quizzes/
- **Decide: do your students code? Separate question from whether you code, and it changes the assignment's structure.**
- Students code → a starter notebook/script with clear TODOs, submitted as their own job
- Students don't code → pre-run then interpret (they get output, not a script to edit), or config-only (they change one input value and click Run All)
- Either way, the graded deliverable becomes reasoning about output, not code correctness

HPC Acclimation, Module 2: Interactive Sessions with idev

HPC ACCLIM. 2

- idev starts an interactive session on the supercomputer — like getting your own temporary terminal on the big machine, instead of submitting a job and waiting
- Useful for testing something quickly before running it at full scale in a batch job
- launch: `idev` (defaults to 30 minutes, single node)
- custom: `idev -p gg -N 2 -n 8 -m 150` (gg queue, 2 nodes, 150 minutes)
- Once allocated, you get a compute-node command prompt for real-time, hands-on work

HPC Acclimation, Module 2: Running Jupyter via TAP

HPC ACCLIM. 2

- Launch an interactive Jupyter session via idev, submitted through the TAP Analysis Portal rather than a raw terminal session
- This is the same idev mechanism just covered — TAP wraps it so the session shows up as a notebook interface your students can use directly
- Hands-on goal today: get one notebook running on Vista end to end before moving to batch submission

HPC Acclimation, Module 3: Submitting Batch Jobs

HPC ACCLIM. 3

Submit with `sbatch myjobscript`. Critical restriction: never request specific compute nodes without TACC approval — the job gets deleted. Let Slurm allocate automatically.

Flag	Purpose	Example
-N	Number of nodes (required)	-N 4
-n	Total MPI tasks	-n 576
-t	Wall clock time (required)	-t 02:00:00
-p	Queue name	-p gh
-o / -e	Output / error file	-o out.log

HPC Acclimation, Module 3: Monitoring & Reading Output

HPC ACCLIM. 3

- Queues available on Vista: gg (up to 32 nodes, 48 hrs), gh (up to 64 nodes, 48 hrs), gh-dev (up to 8 nodes, 2 hrs)
- MPI jobs launch with ibrun, not mpirun — the TACC-specific launcher that reads your Slurm allocation automatically
- Charging model: SUs billed = (# nodes) × max(wall-clock hours, 0.25) × charge rate/node-hour; minimum 15-minute charge regardless of actual runtime
- This is also exactly the kind of output an HPC-literacy exam question can test — see Day 4's exam scaffold

- A fast session — set the direction here, finish independently if time runs short
- Draft a grading scale and category weighting from your assessment types
- Build a reusable assignment rubric template — a shape you'll apply to individual assignments later
- Build an AI-use rubric with separate criteria: coders (did they understand the code their AI coding assistant helped write?), non-coders (can they explain what the generated script does?)

DAY 4

Case Study, Exam Scaffold + Finish the Module

Thursday, July 30

Day 4: Today's Schedule

Time	Session
10:00–10:15	Daily agenda review
10:15–11:30	Case study example
11:30–12:30	FAQs with NotebookLM
12:30–1:30	Lunch
1:30–2:00	Draft your exam scaffold
2:00–2:30	Setting up student allocations: HPC access language for your syllabus
2:30–3:00	Finish building your module
5:00 PM	Citrus Fusion Cuisine (evening)

Case Study Example

- A worked example of a full HPC-based course module, start to finish
- Purpose: ground what you're building this week against a real reference point before you finish your own module this afternoon
- Look for how the example handles the public/private split, the HPC component's scale-first justification, and the coder/non-coder assignment branch

FAQs with NotebookLM

- NotebookLM is an AI research and Q&A tool that grounds its answers in documents you upload to it, rather than general web knowledge
- Use case this week: load your own course materials (syllabus, module docs, program resources) and ask it FAQ-style questions your students are likely to ask
- Good for generating a first-pass FAQ page or office-hours prep sheet grounded in what you've actually written, not generic answers

A midterm scaffold mixing conceptual and applied questions, with an answer key kept in the same private file.

“Draft a midterm exam scaffold for [course topic], covering these outcomes: [paste from learning-outcomes-map.md]. Mix conceptual and applied questions.”

Include at least one HPC-literacy question that doesn't require the student to have written code themselves — e.g. “Here's Slurm job output; what happened, and what would you check next?”

HPC Access Language for Your Syllabus: Timeline

- Students need the same account path you followed pre-event: request at bpccenter.org/current-programs/cbpc-hpc-resources with a .edu email, approval within a few business days
- Build in lead time — request this well before the assignment is due, not the week of, exactly like your own account setup this summer
- MFA (Duo/Okta) setup follows approval, same as your own onboarding
- Program contact for account questions: Dr. Ashley Scruse, Deputy Director (ashley.scruse@morehouse.edu)

HPC Access Language for Your Syllabus: Conduct & Suggested Language

- Students should know before their first login: no compute-intensive work on login nodes, stage data through \$SCRATCH not \$WORK, and accounts can be suspended for impacting other users
- Suggested syllabus paragraph: “This course uses the Morehouse Supercomputing Facility (Vista). Request access by [date] at [link]; approval takes several business days. You are responsible for following TACC's acceptable-use policy — questions go to [contact].”
- This directly supports the HPC-literacy exam question from Module 06 — students who understand the policy context read job output more confidently

- ☐ `course-site/modules/module-XX.md` links to everything: slides, assignment, quiz link (not quiz content)
- ☐ Full prep notes complete in `course-toolkit/lecture-prep-notes/`
- ☐ Quiz with answer key complete in `course-toolkit/quizzes/`
- ☐ Assignment supplementary material complete in `course-site/resources.md`
- ☐ Public file re-checked — no answer keys, grading notes, or private prep content leaked in

DAY 5

Publish, Share, Plan, and Sustain

Friday, July 31

Day 5: Today's Schedule

Time	Session
10:00–10:15	Daily agenda review
10:15–11:00	Publish course site; office hours setup; export to Canva & Blackboard; reuse template
11:15–12:00	Building your 90-day implementation plan
12:30–1:30	Lunch
1:30–2:00	Using NAIRR resources beyond this week
2:00–2:30	Program evaluation, led by Dr. Mi'Kayla Newell
2:30–2:55	Before program ends: final checklist review
2:55–3:00	Closing remarks & group picture
Evening	Beach day

- Before turning anything on, confirm none of the following are present in course-site: answer keys, grading weights, student data (even de-identified samples), private prep notes
- If comfortable with git, a literal diff between the two repos' structure can help spot anything that crossed over by accident
- GitHub Pages is one option for sharing your site, not a requirement — exporting to Google Classroom, Canvas, or Blackboard works just as well

- course-site → Settings → Pages → set source branch (usually main) → save
- Visit the published URL together; click through every page — syllabus, schedule, modules, resources, office hours — confirm nothing 404s and nothing private is visible
- Note what you'll need to update each semester (dates, office hours link, semester-specific policy language) so future-you isn't starting from scratch

- Set up your booking link — Calendly (if created pre-event) or Teams, per your institution's setup
- Write a short policy: format (in-person/virtual), what students should bring, response-time expectations
- Publish both into course-site/office-hours.md

- Export Module 4's slide decks and other visual materials for your LMS
- If your campus LMS admin confirmed Canva–Blackboard LTI is enabled pre-event, walk through the export now
- LTI (Learning Tools Interoperability) is the technical handshake that lets Canva plug directly into Blackboard — slides go straight from Canva into your course page instead of manual download/re-upload
- Not enabled yet? Note it as a pending item for your 90-day plan rather than blocking here

- Confirm both course-site and course-toolkit are set as GitHub template repos (Settings → Template repository)
- “Use this template” produces a clean copy, no shared commit history, ready to rename for your next course or cohort
- **Carries over cleanly: repo structure, rubric templates, the AI-use rubric, this Builder Guide walkthrough itself**
- **Needs rebuilding each time: learning outcomes, curriculum map, actual module content, exam questions**

90-Day Implementation Plan

- Note which toolkit artifacts are still pending, and what department or institutional buy-in you still need
- Connect directly to what carries over vs. rebuilds from the previous session — use that list as your starting checklist
- Be specific: what ships in the Fall semester, and what's realistically a “next cohort” item

Using NAIRR Resources Beyond This Week

- Continued access & community of practice — your access to NAIRR resources doesn't end when the accelerator does
- Join the NAIRR community-of-practice listserv to stay connected with this cohort and past ones
- Optional, self-paced CS-track modules available anytime: Module 11 (Parallel Computing with MPI & OpenMP), Module 12 (Containerization for Reproducibility), and Module 13 (ML Hub for Tapis) — for instructors who want more technical depth than the general track covers
- Module 07 (AI for Grading & Attendance) is also optional and self-paced, for whenever you're ready

Before You Leave: Final Checklist

- ☐ Course site finished and shared however you've chosen — published, exported to your LMS, or both
- ☐ Private repo confirmed still private
- ☐ Program evaluation survey completed
- ☐ Joined the NAIRR community-of-practice listserv
- ☐ Exchanged contact info with your track peers

The Full Builder Guide: All 13 Modules

#	Module	Notes
00–01	Set Up Your Repos & Environment	
02	Learning Outcomes → Curriculum Map	
03	Draft Your Syllabus	
04	Module Builder	3 parts, spans Days 2–4
05	Grading Scale & Rubrics	
06	Exam Scaffold	
07	AI for Grading & Attendance	optional, self-paced
08	Office Hours Setup	
09	Publish Your Course Site	
10	Reuse This Template	
11	Parallel Computing (MPI/OpenMP)	optional, CS track
12	Containerization for Reproducibility	optional, CS track
13	ML Hub for Tapis	optional, CS track